

Exam-wise Notes — Coal India MT (Systems)

Theory of Computation • Compiler Design • Engineering Mathematics • GATE-core depth consolidation

How to read this exam-wise PDF

This covers the topics **unique to Coal India MT (Systems, Paper II)** that the shared umbrella did not teach — plus a consolidated map of the **amber GATE-core depth** flagged across the six umbrella modules.

Depth here is **GATE-level** (Coal India's Paper II tracks the GATE CS syllabus), higher than the umbrella's middle depth.

■ **Green box** = how Coal India / GATE-style PSUs test the topic. ■ **Purple box** = scope/pacing note.

Pair this with the umbrella: shared subjects (OS, DBMS, CN, DS, COA, Algorithms) stay in Modules 1-6; this PDF adds only what's Coal-India-specific.

0 What this PDF covers

Coal India's Systems Paper II is essentially the GATE CS syllabus. Six of its ten heads (Programming/DS, Algorithms, OS, Databases, CN, Digital Logic/COA) are already in your umbrella. This PDF completes the remaining, exam-specific heads:

- **Part A — Theory of Computation (TOC):** automata, grammars, languages, decidability.
- **Part B — Compiler Design:** the phases that turn source code into executable form.
- **Part C — Engineering Mathematics:** discrete math, linear algebra, calculus, probability, numerical methods.
- **Part D — GATE-core depth map:** a checklist of the amber 'go deeper' items across Modules 1-6, so nothing is missed for the harder paper.

■ Tested pattern & focus (PSU / Govt CS-IT-Systems)

Coal India Paper II is objective but **GATE-depth**: expect derivational and numerical questions, not just recall. TOC, Compiler, and Engineering Maths together carry meaningful weight and are pure Coal-India (not in IBPS/DSSSB).

Highest-yield within these: **finite automata & regular languages, parsing (LL/LR), lexical vs syntax phases, and probability + linear algebra** basics.

PART A Theory of Computation

TOC studies **what problems can be computed** and by what abstract machine. It builds a ladder of machines, each more powerful than the last, each recognising a larger class of languages. This **Chomsky hierarchy** is the spine of the subject.

Machine	Recognises	Grammar type	Memory
Finite Automaton (FA)	Regular languages	Type 3 (regular)	None (states only)
Pushdown Automaton (PDA)	Context-free languages	Type 2 (context-free)	One stack
Linear Bounded Automaton	Context-sensitive languages	Type 1	Bounded tape
Turing Machine (TM)	Recursively enumerable	Type 0 (unrestricted)	Infinite tape

A.1 Finite automata & regular languages

- **DFA** (Deterministic FA) — exactly one transition per (state, symbol). **NFA** — may have multiple/ ϵ transitions; DFA and NFA are **equivalent in power** (every NFA converts to a DFA).
- **Regular expressions** describe regular languages; **Kleene's theorem**: RE, DFA, and NFA all define exactly the regular languages.
- **Pumping lemma (regular)** — used to *prove* a language is **not** regular (e.g. $a^n b^n$).
- Closure: regular languages are closed under union, concatenation, star, intersection, complement.

A.2 Context-free languages & PDA

- **Context-Free Grammar (CFG)** — productions of form $A \rightarrow \alpha$. Generates context-free languages (e.g. balanced parentheses, $a^n b^n$).
- **PDA** = FA + a stack; recognises CFLs. Non-deterministic PDAs are more powerful than deterministic ones.
- **Ambiguity** — a grammar is ambiguous if some string has two parse trees. **Normal forms**: CNF (Chomsky) and GNF (Greibach).
- **Pumping lemma (CFL)** — proves a language is not context-free (e.g. $a^n b^n c^n$).

A.3 Turing machines & decidability

- **Turing Machine** — the most powerful model; defines **recursively enumerable** languages. A language is **recursive** (decidable) if a TM always halts with yes/no.
- **Halting problem** — the classic **undecidable** problem: no algorithm can decide, for all programs, whether they halt.
- **Decidable vs undecidable**, and **P vs NP** basics (NP-complete, reductions) — conceptual questions appear.

■ Tested pattern & focus (PSU / Govt CS-IT-Systems)

Most-asked TOC: **DFA/NFA equivalence & construction**, identifying the **language of a regular expression**, **pumping-lemma** use, the **Chomsky hierarchy** table above, and **halting problem = undecidable**.

Numerical/derivational: minimise a DFA, convert NFA→DFA, or count states — practise these.

PART B Compiler Design

A **compiler** translates a whole high-level program into machine/target code, in a fixed sequence of **phases**. Coal India tests the phase order, what each phase does, and parsing in particular.

Phase	Job	Output
1. Lexical analysis	Group characters into tokens; remove whitespace/comments	Tokens (via a scanner/DFA)
2. Syntax analysis (parsing)	Check grammar; build the parse/syntax tree	Parse tree
3. Semantic analysis	Type checking, scope, meaning validation	Annotated tree
4. Intermediate code gen	Produce machine-independent IR	3-address code
5. Code optimization	Improve IR (speed/size) without changing meaning	Optimised IR
6. Code generation	Produce target machine code	Machine code

Two support structures run across phases: the **symbol table** (stores identifiers & attributes) and the **error handler**.

B.1 Front end vs back end

The **front end** (phases 1-4) is source-language dependent and machine-independent; the **back end** (5-6) is machine-dependent. This split lets one front end serve many targets.

B.2 Lexical analysis

The **lexical analyzer (scanner)** reads characters and produces **tokens** (keywords, identifiers, operators, literals). It's built on **regular expressions / finite automata** — directly linking to Part A. A **lexeme** is the actual character string; a **token** is its category.

B.3 Syntax analysis (parsing)

- **Parser** uses a CFG to verify structure. Two families:
- **Top-down parsing** — builds the tree from the root; e.g. **recursive descent**, **LL(1)** (needs a grammar with no left recursion, and left-factored).
- **Bottom-up parsing** — builds from leaves to root; e.g. **LR(0)**, **SLR**, **LALR**, **CLR** (more powerful, table-driven). LALR is used by tools like YACC.
- **Left recursion** and **left factoring** are grammar transformations needed for top-down parsing.

■ Tested pattern & focus (PSU / Govt CS-IT-Systems)

Highest-yield compiler MCQs: **phase order & what each does**, **lexical analyzer uses finite automata/regex**, **token vs lexeme**, and **top-down (LL) vs bottom-up (LR)** parsing distinctions.

Know: parse tree is produced by **syntax analysis**; three-address code is **intermediate** code; symbol table spans all phases.

■ Coverage note / warning

Deep parsing tables (FIRST/FOLLOW sets, LR item construction, SDT attribute evaluation) are the hardest GATE bits. For Coal India's objective paper, prioritise phase concepts + LL/LR distinction first; go to full table construction only if time allows.

PART C Engineering Mathematics

Coal India's Paper II includes the GATE CS mathematics. It's broad; the highest-yield areas for an objective paper are discrete math, linear algebra, and probability.

C.1 Discrete Mathematics

- **Set theory & relations** — operations, equivalence relations, partial orders (POSET), Hasse diagrams.
- **Functions** — injective/surjective/bijective; counting.
- **Propositional & predicate logic** — connectives, truth tables, quantifiers, validity, implication (links to Boolean algebra in Module 5).
- **Combinatorics** — permutations, combinations, pigeonhole principle, inclusion-exclusion, recurrence relations, generating functions.
- **Graph theory** — trees, connectivity, Euler/Hamiltonian paths, colouring (links to graph algorithms in Module 6).
- **Groups & lattices** — basic algebraic structures; lattice as a POSET with lub/glb.

C.2 Linear Algebra

- **Matrices & determinants** — operations, inverse, rank.
- **Systems of linear equations** — consistency, Gaussian elimination.
- **Eigenvalues & eigenvectors** — characteristic equation; a recurring GATE topic.

C.3 Calculus

- **Limits, continuity, differentiability**; maxima/minima.
- **Mean value theorems**, definite/indefinite integrals, and basic convergence.

C.4 Probability & Statistics

- **Probability basics** — conditional probability, Bayes' theorem, independence.
- **Random variables** — expectation, variance; distributions (uniform, binomial, Poisson, normal, exponential).
- **Descriptive stats** — mean, median, mode, standard deviation.

C.5 Numerical Methods

- **Root finding** — bisection, Newton-Raphson.
- **Interpolation** — Newton's forward/backward, Lagrange (also a DSSSB DOE head).
- **Numerical integration** — trapezoidal & Simpson's rules.

■ Tested pattern & focus (PSU / Govt CS-IT-Systems)

For an objective paper, prioritise **probability** (Bayes, expectation), **linear algebra** (eigenvalues, rank, matrix ops), and **discrete math** (counting, logic, relations) — these give the most questions per hour of study.

PART D GATE-core Depth Map (from Modules 1-6)

Every amber '↑ go deeper' box across the umbrella, gathered here as a second-pass checklist. These are shared subjects where Coal India needs more depth than IBPS/DSSSB — study them in the parent module, using this as your tick-list.

Module (topic)	GATE-core depth to add for Coal India
M1 Computer Networks	Nyquist/Shannon capacity; delay components; sliding-window efficiency & throughput; Go-Back-N vs Selective-Repeat window sizes; IP fragmentation/MTU math; Dijkstra/Bellman-Ford routing runs
M2 Operating Systems	Scheduling numericals (incl. RR w/ overhead); Banker's Algorithm (safe sequence); resource-allocation graphs; TLB + paging address arithmetic; page-fault problems; multilevel-feedback queues
M3 DBMS	Relational algebra & tuple calculus; attribute closure & candidate-key finding; lossless-join + dependency-preserving decomposition; 4NF; conflict-serializability (precedence graph); log-based recovery
M4 Prog & DS	Pointers & memory layout; storage classes; recursion tracing/output; AVL rotations; heap build/heapify; tree height/node-count formulas; traversal output problems
M5 Digital Logic + COA	K-maps (4-var, don't-cares); flip-flop excitation & counter design; signed/floating-point (IEEE 754) arithmetic; cache mapping (direct/assoc/set-assoc) + hit-ratio & EAT math; pipelining stages, speedup & hazards; addressing-mode problems
M6 Algorithms + SE	Recurrences (Master theorem, substitution); algorithm tracing & correctness; graph-algorithm runs (Dijkstra/Prim/Kruskal/Floyd-Warshall); amortized analysis; NP-completeness; COCOMO, function points, cyclomatic complexity

■ Coverage note / warning

This depth is **only** for Coal India (and GATE-style PSUs). For IBPS SO IT and DSSSB TGT, the umbrella's middle depth is sufficient — do not spend time here for those two exams.

Rapid recap — Coal India exam-specific

Area	Nail these
TOC	Chomsky hierarchy: FA/regular → PDA/CFL → LBA/CSL → TM/REL; DFA $\equiv$ NFA; pumping lemma proves NON-membership; halting problem undecidable
Compiler	Phases: lexical → syntax → semantic → intermediate → optimize → codegen; front end vs back end; scanner uses FA/regex; token vs lexeme; LL(top-down) vs LR(bottom-up); symbol table spans all
Discrete Math	Sets/relations/POSET; logic & quantifiers; combinatorics (perm/comb, pigeonhole); graph theory; groups/lattices
Linear Algebra	Matrix ops, rank, inverse; solving linear systems; eigenvalues/eigenvectors
Probability	Conditional prob, Bayes; expectation & variance; binomial/Poisson/normal
Numerical	Newton-Raphson, bisection; interpolation (Newton/Lagrange); trapezoidal/Simpson
Depth map	Work the Part-D checklist inside Modules 1-6 as a GATE second pass

✓ Where this sits

This completes CS/IT coverage for **Coal India MT (Systems)**: umbrella Modules 1-6 (shared core, worked to GATE depth via Part D) + this PDF's TOC, Compiler, and Engineering Mathematics.

Next exam-wise PDFs in the plan: **IBPS** (Cloud + Cyber depth) and **DSSSB** (Java/.NET/web/graphics/MIS + IT fundamentals/Python/Pandas), plus the shared **Emerging Tech & Security** PDF. Q&A sets follow your walkthroughs.